17-Executor组件：Tomcat如何扩展Java线程池？

在开发中我们经常会碰到“池”的概念，比如数据库连接池、内存池、线程池、常量池等。为什么需要“池”呢？程序运行的本质，就是通过使用系统资源（CPU、内存、网络、磁盘等）来完成信息的处理，比如在JVM中创建一个对象实例需要消耗CPU和内存资源，如果你的程序需要频繁创建大量的对象，并且这些对象的存活时间短，就意味着需要进行频繁销毁，那么很有可能这部分代码会成为性能的瓶颈。

而“池”就是用来解决这个问题的，简单来说，对象池就是把用过的对象保存起来，等下一次需要这种对象的时候，直接从对象池中拿出来重复使用，避免频繁地创建和销毁。在Java中万物皆对象，线程也是一个对象，Java线程是对操作系统线程的封装，创建Java线程也需要消耗系统资源，因此就有了线程池。JDK中提供了线程池的默认实现，我们也可以通过扩展Java原生线程池来实现自己的线程池。

同样，为了提高处理能力和并发度，Web容器一般会把处理请求的工作放到线程池里来执行，Tomcat扩展了原生的Java线程池，来满足Web容器高并发的需求，下面我们就来学习一下Java线程池的原理，以及Tomcat是如何扩展Java线程池的。

Java线程池

简单的说，Java线程池里内部维护一个线程数组和一个任务队列，当任务处理不过来的时，就把任务放到队列里慢慢处理。

ThreadPoolExecutor

我们先来看看Java线程池核心类ThreadPoolExecutor的构造函数，你需要知道ThreadPoolExecutor是如何使用这些参数的，这是理解Java线程工作原理的关键。

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          ThreadFactory threadFactory,
                          RejectedExecutionHandler handler)
```

每次提交任务时，如果线程数还没达到核心线程数**corePoolSize**，线程池就创建新线程来执行。当线程数达到**corePoolSize**后，新增的任务就放到工作队列**workQueue**里，而线程池中的线程则努力地从**workQueue**里拉活来干，也就是调用poll方法来获取任务。

如果任务很多，并且**workQueue**是个有界队列，队列可能会满，此时线程池就会紧急创建新的临时线程来救场，如果总的线程数达到了最大线程数**maximumPoolSize**，则不能再创建新的临时线程了，转而执行拒绝策略**handler**，比如抛出异常或者由调用者线程来执行任务等。

如果高峰过去了，线程池比较闲了怎么办？临时线程使用poll（**keepAliveTime, unit**）方法从工作队列中拉活干，请注意poll方法设置了超时时间，如果超时了仍然两手空空没拉到活，表明它太闲了，这个线程会被销毁回收。

仅供个人学习 请勿传播 用户标记 mannix小查理

那还有一个参数**threadFactory**是用来做什么的呢？通过它你可以扩展原生的线程工厂，比如给创建出来的线程取个有意义的名字。

FixedThreadPool/CachedThreadPool

Java提供了一些默认的线程池实现，比如FixedThreadPool和CachedThreadPool，它们的本质就是给ThreadPoolExecutor设置了不同的参数，是定制版的ThreadPoolExecutor。

```
public static ExecutorService newFixedThreadPool(int nThreads) {
    return new ThreadPoolExecutor(nThreads, nThreads,
                                  0L, TimeUnit.MILLISECONDS,
                                 new LinkedBlockingQueue<Runnable>());
}

public static ExecutorService newCachedThreadPool() {
    return new ThreadPoolExecutor(0, Integer.MAX_VALUE,
                                  60L, TimeUnit.SECONDS,
                                  new SynchronousQueue<Runnable>());
}
```

从上面的代码你可以看到：

- **FixedThreadPool有固定长度（nThreads）的线程数组**，忙不过来时会把任务放到无限长的队列里，这是因为**LinkedBlockingQueue默认是一个无界队列**。
- **CachedThreadPool的maximumPoolSize参数值是**`Integer.MAX_VALUE`，因此它对线程个数不做限制，忙不过来时无限创建临时线程，闲下来时再回收。它的任务队列是**SynchronousQueue**，表明队列长度为0。

Tomcat线程池

跟FixedThreadPool/CachedThreadPool一样，Tomcat的线程池也是一个定制版的ThreadPoolExecutor。

定制版的ThreadPoolExecutor

通过比较FixedThreadPool和CachedThreadPool，我们发现它们传给ThreadPoolExecutor的参数有两个关键点：

- 是否限制线程个数。
- 是否限制队列长度。

对于Tomcat来说，这两个资源都需要限制，也就是说要对高并发进行控制，否则CPU和内存有资源耗尽的风险。因此Tomcat传入的参数是这样的：

```
//定制版的任务队列
taskqueue = new TaskQueue(maxQueueSize);
```

```
//定制版的线程工厂
TaskThreadFactory tf = new TaskThreadFactory(namePrefix,daemon,getThreadPriority());

//定制版的线程池
executor = new ThreadPoolExecutor(getMinSpareThreads(), getMaxThreads(), maxIdleTime, TimeUnit.MILLISECONDS
```

你可以看到其中的两个关键点：

- Tomcat有自己的定制版任务队列和线程工厂，并且可以限制任务队列的长度，它的最大长度是maxQueueSize。
- Tomcat对线程数也有限制，设置了核心线程数（minSpareThreads）和最大线程池数（maxThreads）。

除了资源限制以外，Tomcat线程池还定制自己的任务处理流程。我们知道Java原生线程池的任务处理逻辑比较简单：

1. 前corePoolSize个任务时，来一个任务就创建一个新线程。
2. 后面再来任务，就把任务添加到任务队列里让所有的线程去抢，如果队列满了就创建临时线程。
3. 如果总线程数达到maximumPoolSize，**执行拒绝策略。**

Tomcat线程池扩展了原生的ThreadPoolExecutor，通过重写execute方法实现了自己的任务处理逻辑：

1. 前corePoolSize个任务时，来一个任务就创建一个新线程。
2. 再来任务的话，就把任务添加到任务队列里让所有的线程去抢，如果队列满了就创建临时线程。
3. 如果总线程数达到maximumPoolSize，**则继续尝试把任务添加到任务队列中去。**
4. **如果缓冲队列也满了，插入失败，执行拒绝策略。**

观察Tomcat线程池和Java原生线程池的区别，其实就是在第3步，Tomcat在线程总数达到最大数时，不是立即执行拒绝策略，而是再尝试向任务队列添加任务，添加失败后再执行拒绝策略。那具体如何实现呢，其实很简单，我们来看一下Tomcat线程池的execute方法的核心代码。

```
public class ThreadPoolExecutor extends java.util.concurrent.ThreadPoolExecutor {

  ...

  public void execute(Runnable command, long timeout, TimeUnit unit) {
      submittedCount.incrementAndGet();
      try {
          //调用Java原生线程池的execute去执行任务
          super.execute(command);
      } catch (RejectedExecutionException rx) {
         //如果总线程数达到maximumPoolSize，Java原生线程池执行拒绝策略
          if (super.getQueue() instanceof TaskQueue) {
              final TaskQueue queue = (TaskQueue)super.getQueue();
              try {
                  //继续尝试把任务放到任务队列中去
                  if (!queue.force(command, timeout, unit)) {
                      submittedCount.decrementAndGet();
                      //如果缓冲队列也满了，插入失败，执行拒绝策略。
                      throw new RejectedExecutionException("...");
                  }
              }
```

仅供个人学习 请勿传播 用户标记 mannix小查理

```
            }
        }
}
```

从这个方法你可以看到，Tomcat线程池的execute方法会调用Java原生线程池的execute去执行任务，如果总线程数达到maximumPoolSize，Java原生线程池的execute方法会抛出RejectedExecutionException异常，但是这个异常会被Tomcat线程池的execute方法捕获到，并继续尝试把这个任务放到任务队列中去；如果任务队列也满了，再执行拒绝策略。

定制版的任务队列

细心的你有没有发现，在Tomcat线程池的execute方法最开始有这么一行：

```
submittedCount.incrementAndGet();
```

这行代码的意思把submittedCount这个原子变量加一，并且在任务执行失败，抛出拒绝异常时，将这个原子变量减一：

```
submittedCount.decrementAndGet();
```

其实Tomcat线程池是用这个变量submittedCount来维护已经提交到了线程池，但是还没有执行完的任务个数。Tomcat为什么要维护这个变量呢？这跟Tomcat的定制版的任务队列有关。Tomcat的任务队列TaskQueue扩展了Java中的LinkedBlockingQueue，我们知道LinkedBlockingQueue默认情况下长度是没有限制的，除非给它一个capacity。因此Tomcat给了它一个capacity，TaskQueue的构造函数中有个整型的参数capacity，TaskQueue将capacity传给父类LinkedBlockingQueue的构造函数。

```
public class TaskQueue extends LinkedBlockingQueue<Runnable> {

  public TaskQueue(int capacity) {
      super(capacity);
  }
  ...
}
```

这个capacity参数是通过Tomcat的maxQueueSize参数来设置的，但问题是默认情况下maxQueueSize的值是`Integer.MAX_VALUE`，等于没有限制，这样就带来一个问题：当前线程数达到核心线程数之后，再来任务的话线程池会把任务添加到任务队列，并且总是会成功，这样永远不会有机会创建新线程了。

为了解决这个问题，TaskQueue重写了LinkedBlockingQueue的offer方法，在合适的时机返回false，返回false表示任务添加失败，这时线程池会创建新的线程。那什么是合适的时机呢？请看下面offer方法的核心

源码：

```
public class TaskQueue extends LinkedBlockingQueue<Runnable> {

  ...
   @Override
  //线程池调用任务队列的方法时，当前线程数肯定已经大于核心线程数了
  public boolean offer(Runnable o) {

      //如果线程数已经到了最大值，不能创建新线程了，只能把任务添加到任务队列。
      if (parent.getPoolSize() == parent.getMaximumPoolSize())
          return super.offer(o);

      //执行到这里，表明当前线程数大于核心线程数，并且小于最大线程数。
      //表明是可以创建新线程的，那到底要不要创建呢？分两种情况：

      //1. 如果已提交的任务数小于当前线程数，表示还有空闲线程，无需创建新线程
      if (parent.getSubmittedCount()<=(parent.getPoolSize()))
          return super.offer(o);

      //2. 如果已提交的任务数大于当前线程数，线程不够用了，返回false去创建新线程
      if (parent.getPoolSize()<parent.getMaximumPoolSize())
          return false;

      //默认情况下总是把任务添加到任务队列
      return super.offer(o);
  }

}
```

从上面的代码我们看到，只有当前线程数大于核心线程数、小于最大线程数，并且已提交的任务个数大于当前线程数时，也就是说线程不够用了，但是线程数又没达到极限，才会去创建新的线程。这就是为什么Tomcat需要维护已提交任务数这个变量，它的目的就是**在任务队列的长度无限制的情况下，让线程池有机会创建新的线程**。

当然默认情况下Tomcat的任务队列是没有限制的，你可以通过设置maxQueueSize参数来限制任务队列的长度。

本期精华

池化的目的是为了避免频繁地创建和销毁对象，减少对系统资源的消耗。Java提供了默认的线程池实现，我们也可以扩展Java原生的线程池来实现定制自己的线程池，Tomcat就是这么做的。Tomcat扩展了Java线程池的核心类ThreadPoolExecutor，并重写了它的execute方法，定制了自己的任务处理流程。同时Tomcat还实现了定制版的任务队列，重写了offer方法，使得在任务队列长度无限制的情况下，线程池仍然有机会创建新的线程。

课后思考

请你再仔细看看Tomcat的定制版任务队列TaskQueue的offer方法，它多次调用了getPoolSize方法，但是这个方法是有锁的，锁会引起线程上下文切换而损耗性能，请问这段代码可以如何优化呢？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得

仅供个人学习 请勿传播 用户标记 mannix小童鞋

记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。

